1) merge
2) persist
3) remove

Spring boot

1) IOC (Spring)

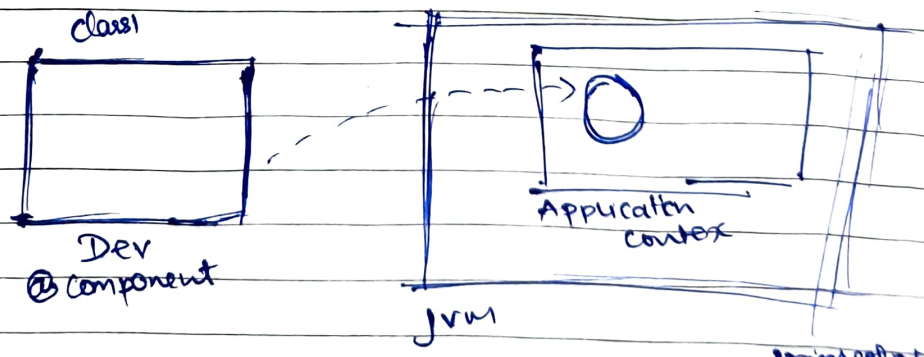
i) (inversion of control.
↳ spring took all the responsible for creation of object (so, no more headache).

ii) Dependency injection.
(Design pattern) and IOC (principal).

Springboot.
↳ embeded tomcat, no configuration.
all things are Managed by springboot.

1) dependency injectio in springboot.

every objects are stored in dvm But in spring there is called spring container or IOC container or Application context.



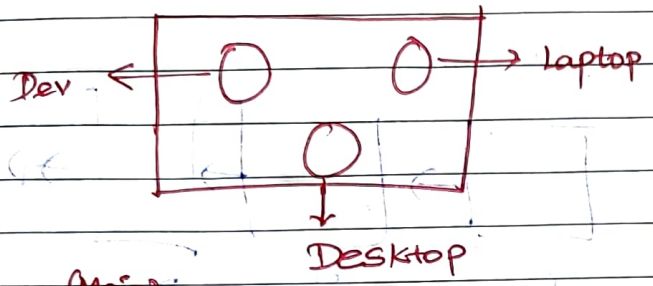

```
ApplicationContext c = SpringApplication.run(.class);
Dev obj = c.getBean(Dev.class)
obj.builder();
```

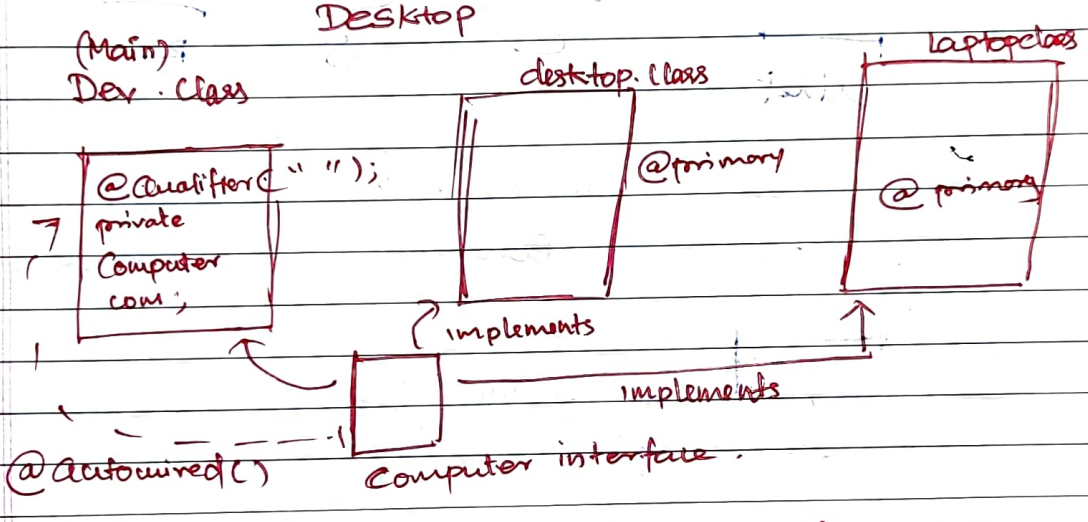
2) Autowired (Connected). field injection.

Setter injection ↰ ↱ Construction injection.

Dev ⟷ Laptop.

↳ autowired





1) @autowired 2) @qualifer 3) primary.

3) Spring without boot.

↳ Maven archtype → dependcy.

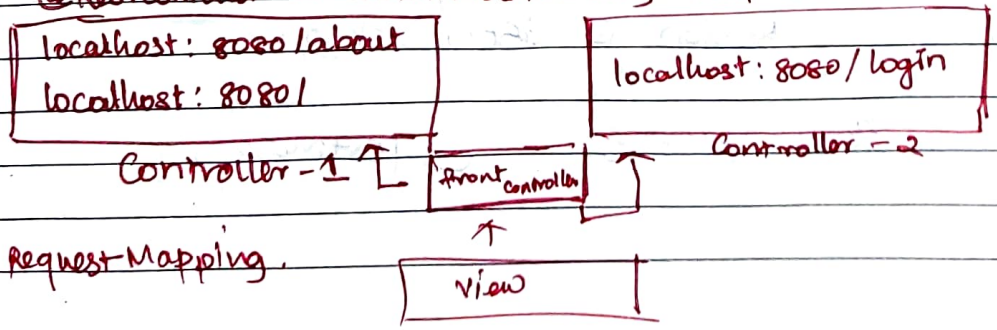
4) Spring Boot web.

↳ add 1) spring stater 2) Spring web

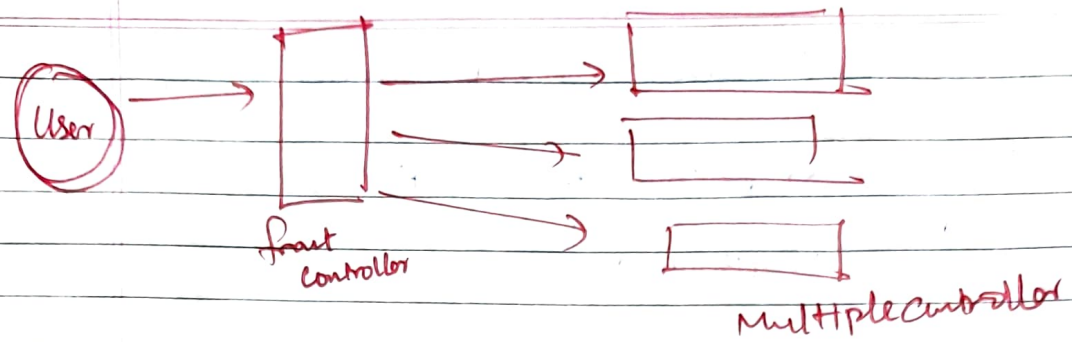
3) Spring DevTool 4) My SQL Driver.

*- In topic i have learn about front controller to.

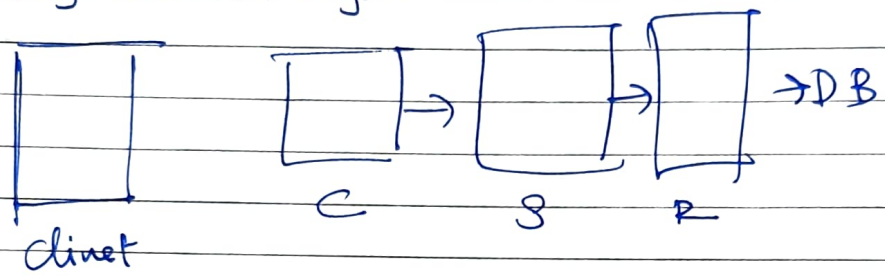
@RestController → return data only and Controller → pages.



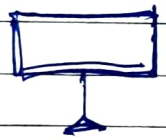
∴ its very simple



5) Spring MVC and Layer:



product Model | productcontroller Controlle | productservice Service.

6)  Spring web HTTP Methods GET and POST.

REST API → HTTP Method

CRUD. { GET and POST, Delete
PUT.

404 → Not found || 505 → Server.

@RequestMapping. 2) @pathvariable
↳ Get value from URL.

*) JSON ⟷ Java object in URL
Reason is ~~JSON~~ Jackson.

@GETMapping @postmapping.

Adhi@51047

→ get all hole object.

@Requestbody → use to post the JSON

7) Delet and update

8) JPA → Repostory (talk to DB).
ORM → object Relationship Mapping.
Java persistant Api. = JPA.
(Spring Data JPA). for Now H2. (H2Database
after setting up → set Spring.datasource.url = Jdbc:h2:mem
: databasename
s.driverclassName = org.h2.Driver || JPA s.show-sql = true.
classname, primary id.

9) in Repository extends JpaRepository<Product, Integer>
@Entity gives for data class. and @Id for primary key.

10) Set up Project:
1) application properietes
2) product → id, name, desc, brand, price, Category
release Date → Date, available, Quantity

11) ResponseEntity<> → check status. Handle status.
CORS = CROSSOrigin, → URL connect Btw FE and BE

12) GET finished JSON format (shape) => dd-MM-YYY.

13) POST (add product).
1) RequestPart vs Requestbody.
2) Save

MultipartFile imagefile.

16) Search → JPQL - classname and fieldname.
@Query. select p from product p where "+
"p.name like '%:keyword%'") allhome

X. Spring Security

29) OWASP → open web Application Security Project.

every 4 year realse security risk.

Dependies → Spring security.

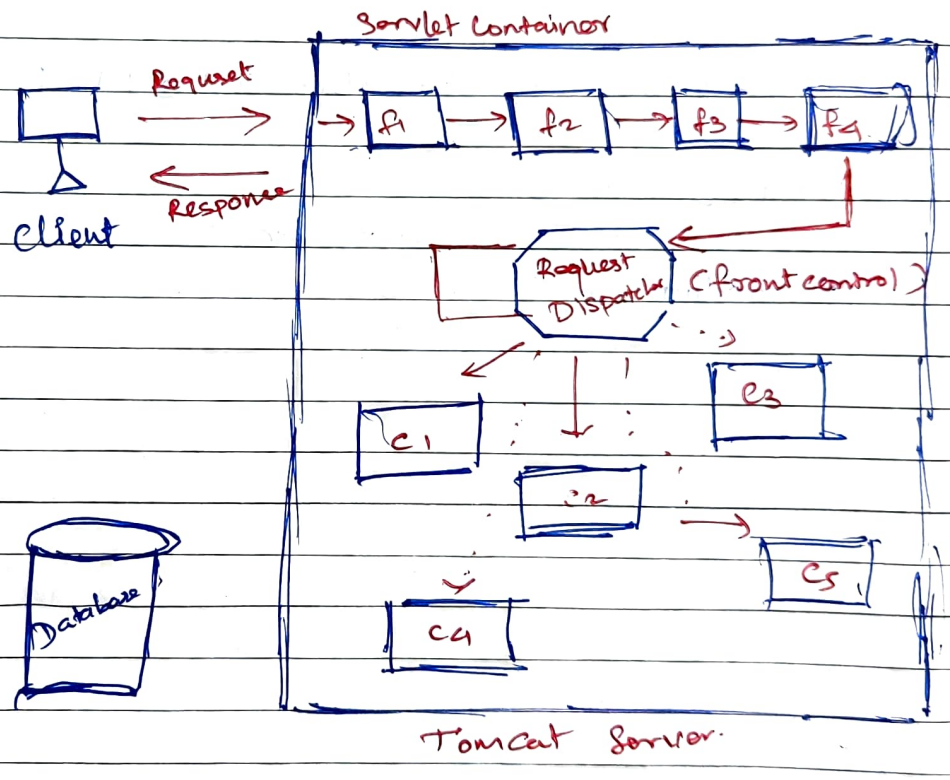
Login → username and password.

① usernamepasswordAuthenticationfilter.

~~Asuat~~ Spring.security.user.name = Asuath

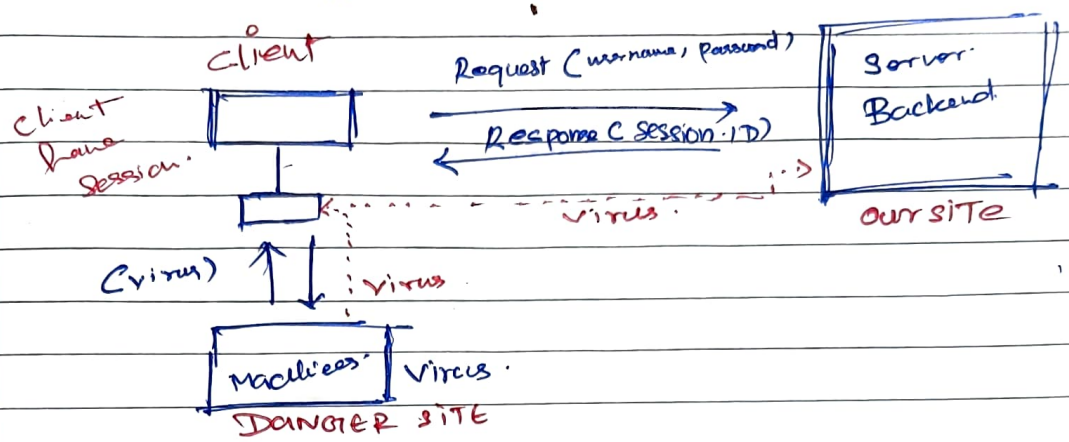
Spring.security.user.password

= telusko.



31) CSRF Token: Cross site Request forgery.

How it is happen: ?



for solve this problem, we are introduction: CSRF Token.

Authorization
Headers(11) → Key: X-CRSF

32) Custom configuration.
@Configuration @EnableWebSecurity.
@Bean for securityfilterchain
HttpSecurity → return obj. Security filterchain.
This all for Bypass security
↳ (Buying Lock But not lock properly.)

if your Make http is stateless, there is No Need of session. Same Session.
SessionManagement (Session. SessionCreationPolicy
(↓. STATELESS)

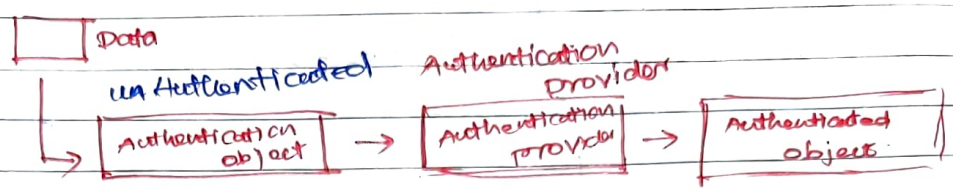
1) Securityfilterchain.
i) csrf disable (customizer).
ii) authorizeHttpRequests. → (authenticated) for
Not allow All request.
HttpBasic → return string value in Postman.
Hi SessionManagement (Session. SessionCreate
↳ changing session.

33) Verify user from Database.

@Bean → UserDetailsService

1) InMemoryUserDetailsManager

User.username().password().roles()



Authentication Providers.

1) Set dependencies.
2) application DB name and password.
3) Authentication provider.
   ↳ DaoAuthentication
      ↳ passwordEncoder, userDetailsService

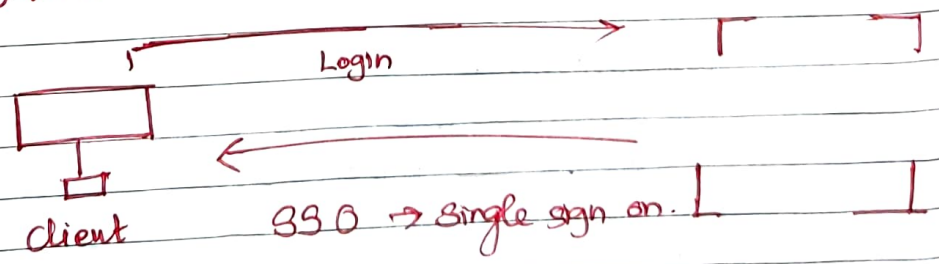
But userDetailsService is Interface, so we can create class.

34) BCrypt

plain → hash1 → hash2.

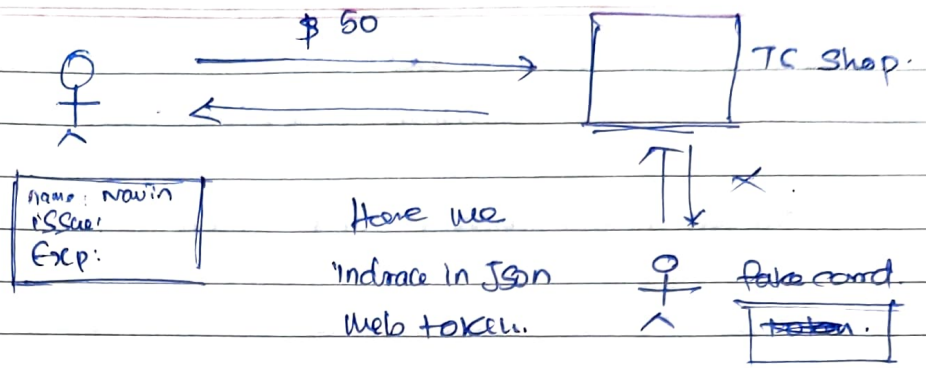
BCryptEncoder, .encode.

35) Jwt (Json web token).



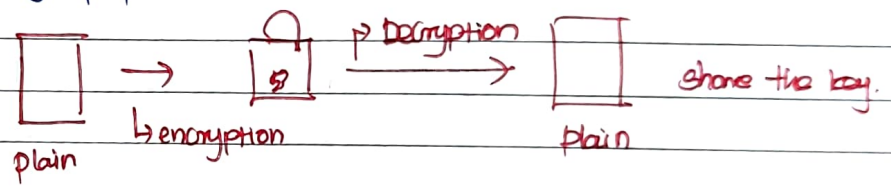
SSO → Single sign on.

it's like Hostel Mess / PG Mess.



Encryption and decryption. alternate key called.
Asymmetric (Data Signature).

36) Cryptography



1) Symmetric key

(key1) A ⇄ B(key1)

have same key.

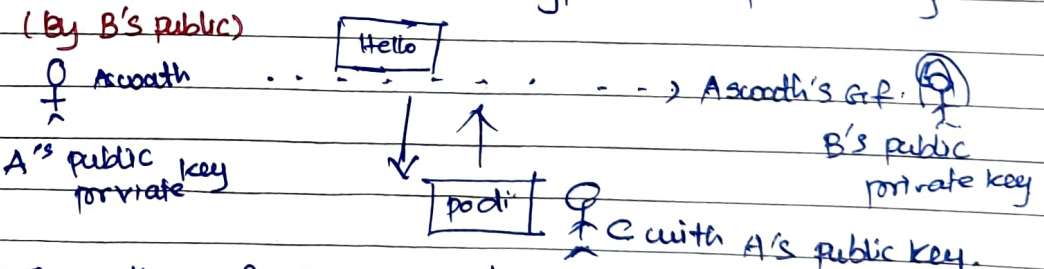
2) Asymmetric key.

D ⇄ A ⇄ B

Encryption → private key.

Decrypted → public key

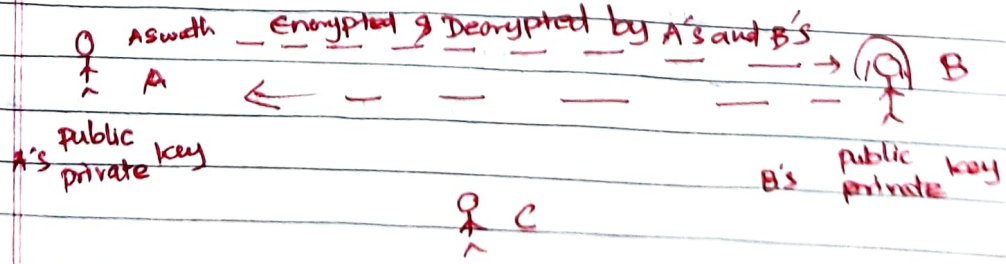
(By B's public)



Then How find it is belongs to whom?

↳ Digital Signature.

Double encryption and decryption.



Setting up project for JWT.

1) JWT (JJWT). Jsonwebtoken API, JJWT - imple

AuthenticationManager for JWT.

Alike Authentication provider → AuthenticatioManager.

1) Configuration. authenticatioManager().

2) Controller (/login) → userService. Verify (user).

3) UserService: Verify → Authentication (Authenticate)
if (isAuthenticated)
return JwtService. generateToken (Username getName())
(from dependency)

4) JwtService
generatetoken
↳ Map<String, object> claims = new HashMap();
return Jwts. build
· claims()
· add (claims)
· Subject (name)
· IssueAt (new Date (S. CT()))
· Iss expiration (new Date (S.CT.()+m))
· and ()(JwtBuilder)
· signwith (getkey())
· Compact();

getkey() : byte[] kB = Decoders. BASE64. decode (Secretkey);
return keys. hmacShakeyFor (kB);

Secretkey = " "
Constructor for JwtService.
KeyGenerator kG = KeyGenerator. getInstance ("HmacSHA256")
SecretKey SK = kG. generateKey();
Secretkey = Base64. getEncoder(). encodeToString (SK. getEncoded());

Validation :

UPAF → User password authentication filter.
1) Jwt is must be first Filter.

Sending Token with Auth token. !
auth Header → request.getHeader
token → get
username → get
and also get username from Database.
By get Bean from ApplicationContext.

39. OAuth2. (give option to another security).

OAuth2 client.
client id and client-Secrets. By using
google cloud console → Api.